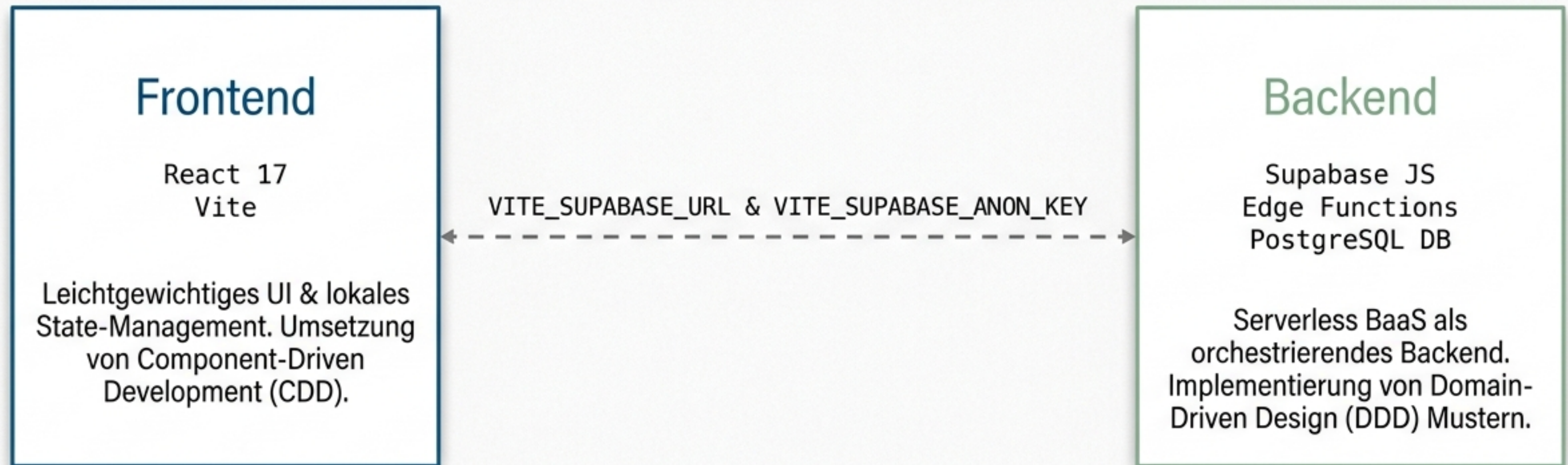


Enterprise-Muster in einer modernen BaaS-Umgebung

Analyse der Architektur: ukondert/test-reactapp-with-supabase

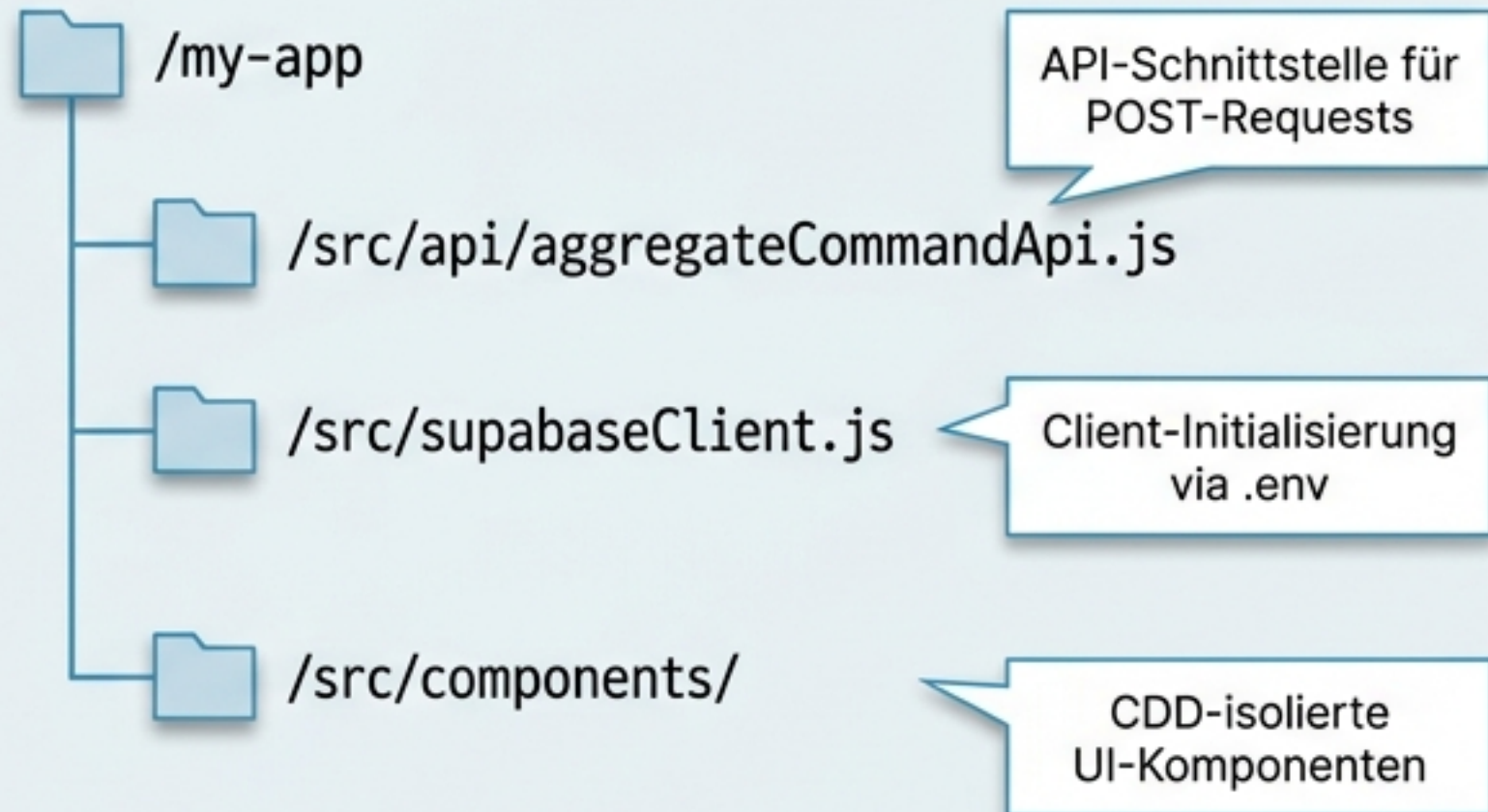


Das Ziel: Skalierbare Software-Muster – spezifisch DDD und CDD – in einer ressourcenschonenden, serverlosen Architektur am Beispiel einer digitalen Schulbibliothek zu implementieren.

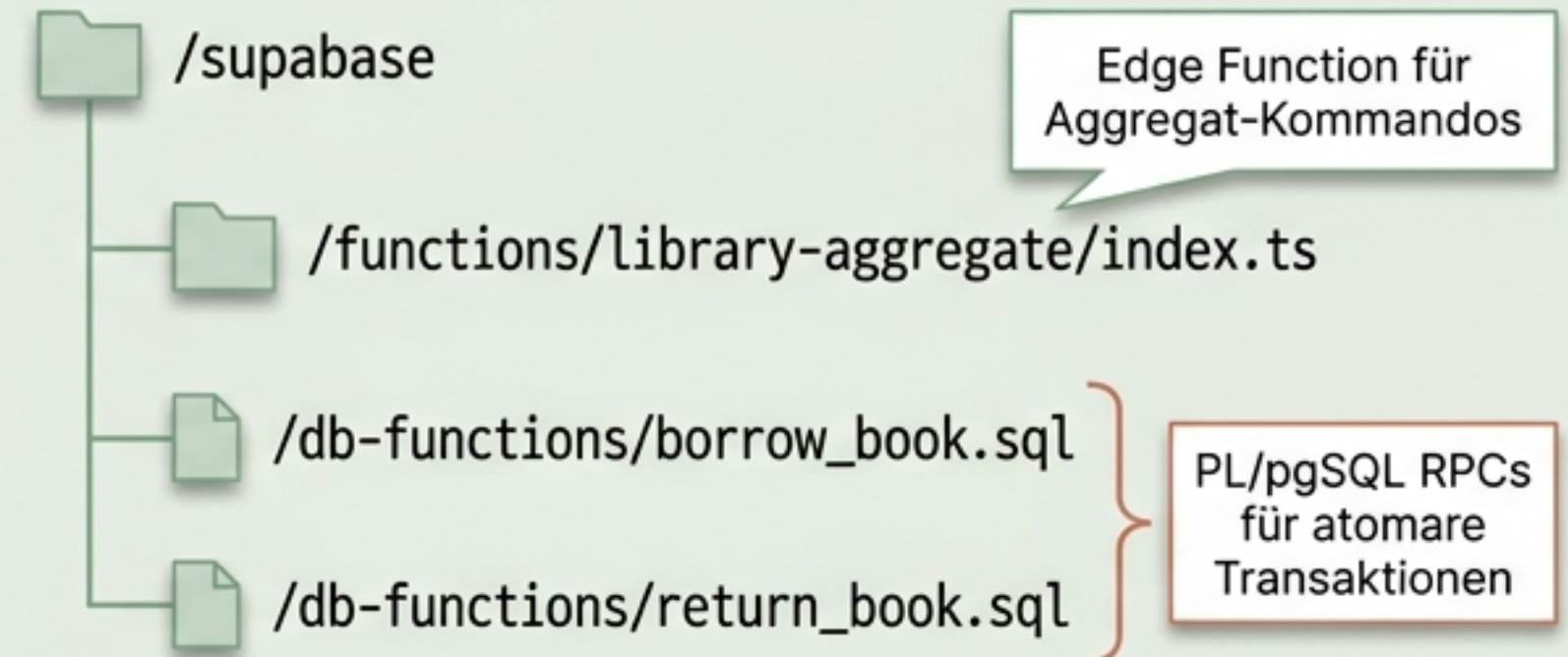
Die physische Trennung von Frontend und Backend-Logik

Eine saubere Architektur beginnt im Dateisystem. Das Repository trennt die UI strikt von der Geschäftslogik durch zwei isolierte Hauptverzeichnisse.

Zone 1: Client Layer



Zone 2: Infrastructure Layer



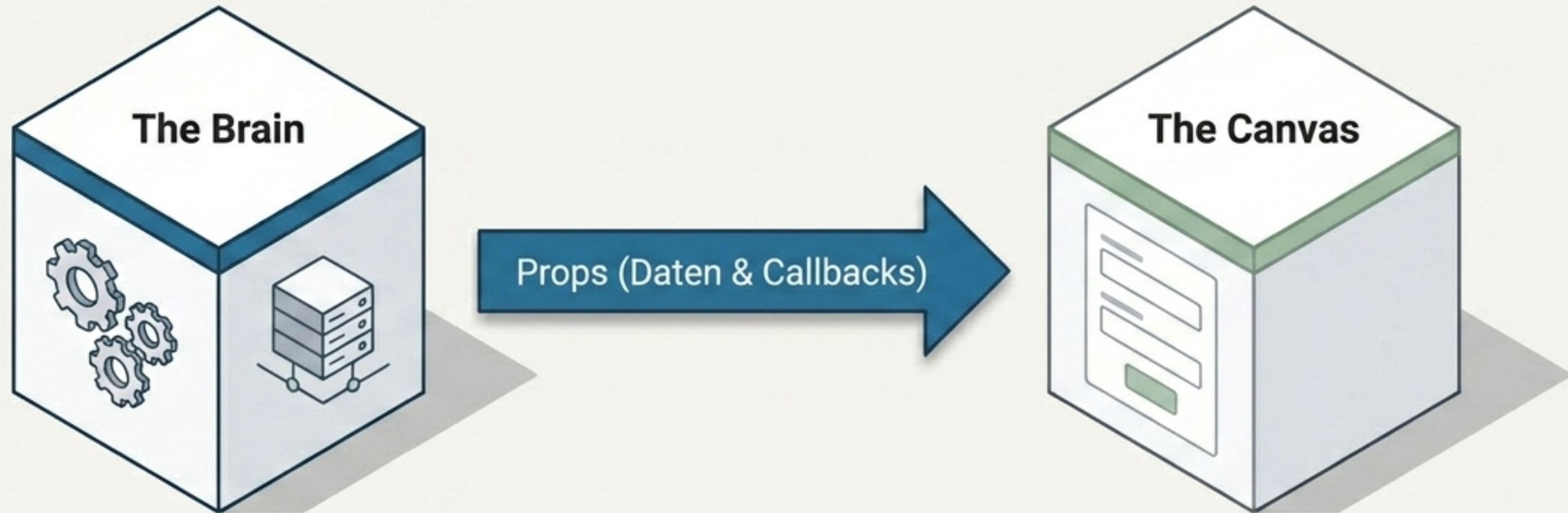
Domain-Driven Design in einer Serverless-Architektur

DDD erfordert keinen monolithischen Server. In diesem Setup verlagern sich die Domänengrenzen: Edge Functions übernehmen die Orchestrierung, während relationale Datenbankfunktionen die Integrität des Aggregats garantieren.

Klassischer Layer (z.B. Spring Boot)	Supabase Äquivalent	Architektonische Funktion
Controller	Edge Function (HTTP Listener)	Nimmt den Request entgegen, validiert das JSON-Format.
Application Service	Edge Function Logic	Orchestriert den Ablauf: Lädt Daten, ruft Domänen-Logik auf.
Repository (Fetch)	Supabase Client (PostgREST)	<code>supabase.from('table').select(...)</code> ersetzt das Fetch-Interface.
Aggregate Root / Logic	Domain Logic / PL/pgSQL	Die eigentliche Invarianten-Prüfung (z. B. Limit prüfen).
Persistence (Save)	RPC (Postgres Function)	Garantiert die atomare Speicherung des gesamten Aggregates.

Component-Driven Development durch isolierte UI-Schichten

Die UI-Hierarchie ist strikt getrennt. Komplexe Abhängigkeiten (State, Supabase-APIs) verbleiben im Container, während die eigentliche Darstellung als reines 'Atom' isoliert und testbar bleibt.



BookCommandFormContainer.jsx

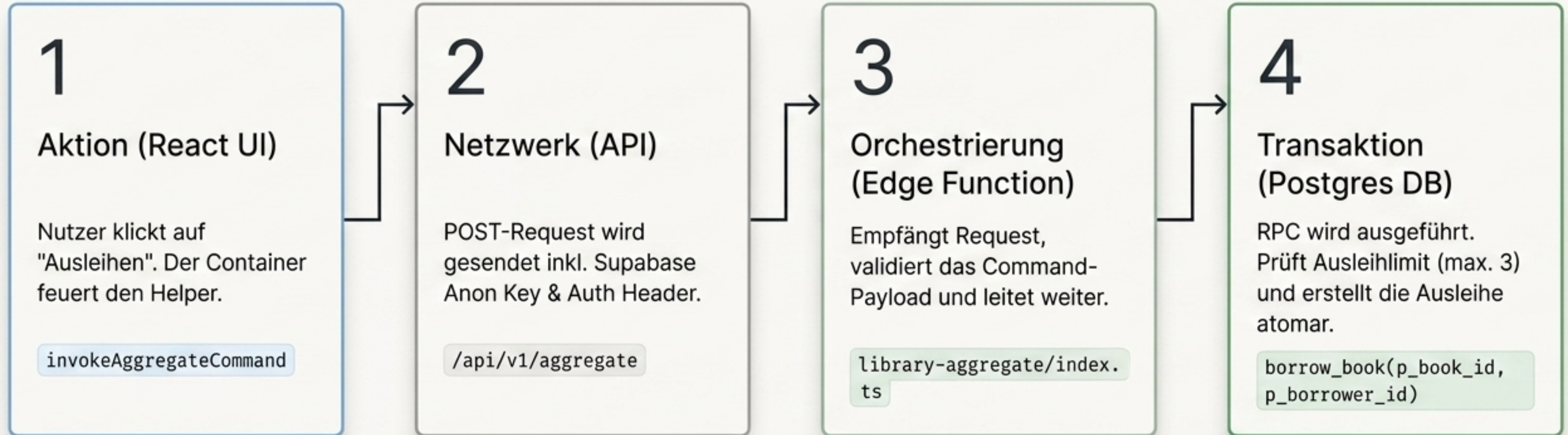
- Smart Component (Container)
- Verwaltet den lokalen Zustand (State)
- Orchestriert die Geschäftslogik
- Ruft `invokeAggregateCommand` auf

BookCommandForm.jsx

- Dumb Component (Presentational)
- Rein visuelle Darstellung
- Keine eigene API-Logik
- Erhält Daten und Callbacks ausschließlich via Props

Der Datenfluss: Der Lebenszyklus eines Commands

Jeder Schritt im "Buch ausleihen" Prozess hat eine exakte Verantwortlichkeit. Die Domänenlogik ist sicher im Backend gekapselt und kann vom Client nicht umgangen werden.



Architektonische Bewertung und Best Practices

Die Kombination aus React-CDD und Supabase-DDD schafft ein robustes Fundament für skalierbare Apps. Es verschiebt die Komplexität vom Monolithen in die Infrastruktur – ein Architekturtausch, der sich bei wachsenden Projekten stark auszahlt.

Stärken (Architekturvorteile)

- **Strikte Trennung**

UI-Entwickler und Datenbank-Ingenieure können durch definierte API-Verträge völlig unabhängig arbeiten.

- **Atomare Sicherheit**

Durch PL/pgSQL RPCs sind Dateninkonsistenzen (z. B. eine Überbuchung von Büchern) auf Datenbankebene ausgeschlossen.

- **Hohe Testbarkeit**

CDD ermöglicht isoliertes Testen der UI-Formulare, während Edge Functions getrennt per API getestet werden.

Trade-offs (Inhärente Risiken)

- **Setup-Komplexität**

Erfordert striktes Management lokaler `.env` Variablen, der Supabase CLI und Access Tokens für Deployments.

- **Verteilte Logik**

Die Fehlersuche und das Debugging erstrecken sich über Client, Netzwerk, Edge Function und Datenbank-Trigger.